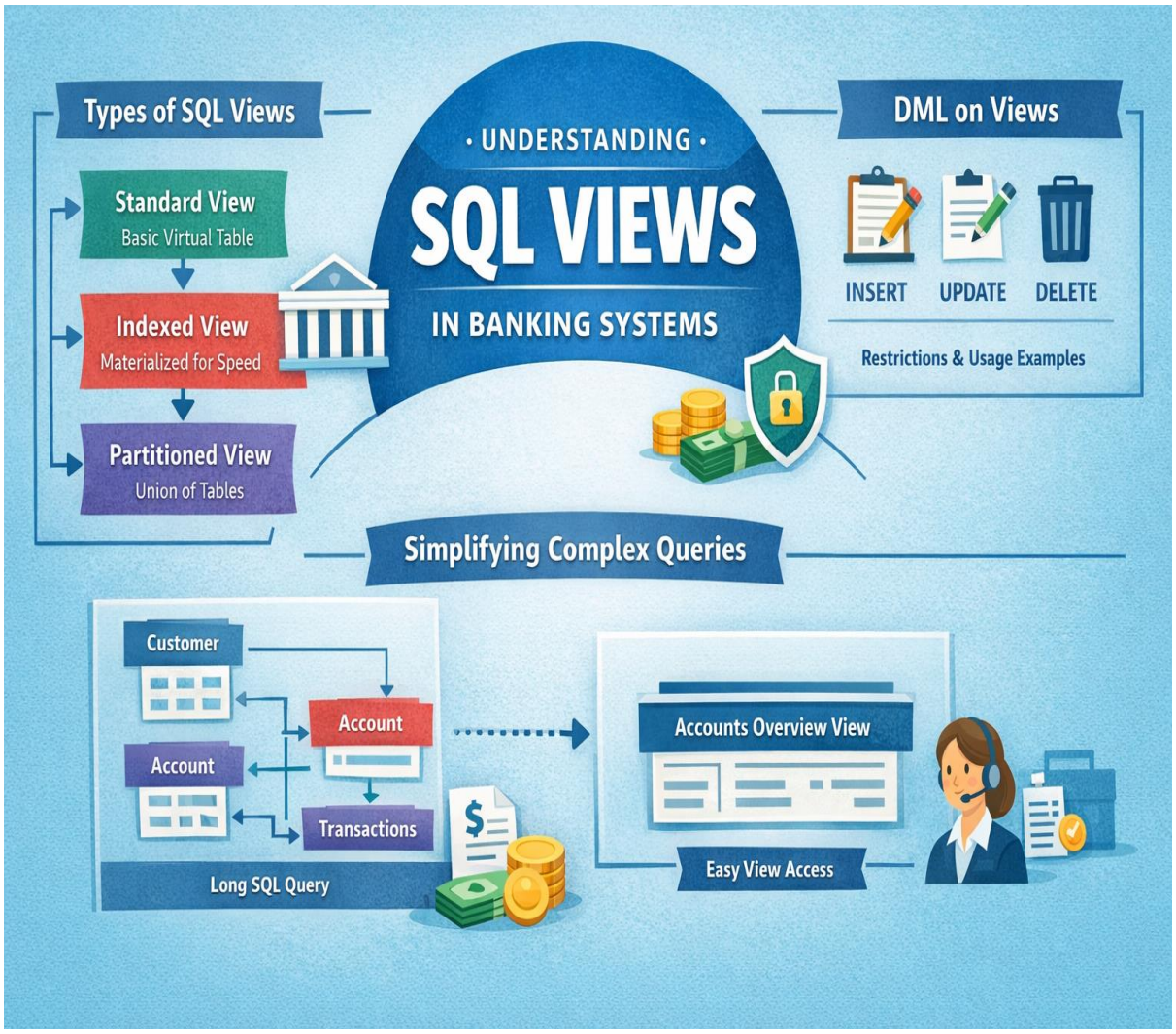
SQL Views | Research & Implementation Task



Part 1: Research & Documentation

Types of Views in SQL Server

A **view** in SQL Server is a virtual table created using a SELECT statement. It does not store data itself but displays data from one or more tables. Views are mainly used for **simplification, security, and data abstraction**.

1. Standard View

What is it?

A **Standard View** is a virtual table that displays data from one or more tables using a stored SELECT query. The data is retrieved dynamically every time the view is accessed.

Key Differences from Other View Types

- Does **not store data physically**
- Executes the underlying query each time it is used
- Simpler than indexed and partitioned views

Real-Life Use Case

University System
A standard view can show student names, courses, and grades without allowing users to access the original tables.

Example:
A view that shows student information for lecturers while hiding sensitive data like passwords or national IDs.

Limitations and Performance Considerations

- Slower than indexed views for large datasets
- Performance depends on the complexity of the underlying query
- Cannot significantly improve query speed by itself

2. Indexed View

What is it?

An **Indexed View** is a view that has a **unique clustered index**, which means the result set is **stored physically** in the database.

Key Differences from Other View Types

- Stores data physically (unlike standard views)
- Improves performance for complex queries
- Requires `SCHEMABINDING`
- Has more creation restrictions

Real-Life Use Case

Banking System
An indexed view can store daily total transaction amounts per account to speed up reporting and auditing.

Example:
A view that calculates total deposits per customer for fast balance summaries.

Limitations and Performance Considerations

- Slower insert, update, and delete operations
- Requires more storage space
- Has strict rules (no `COUNT(*)`, limited joins, schema binding required)
- Best suited for **read-heavy** systems

3. Partitioned View (Union View)

What is it?

A **Partitioned View** combines data from multiple tables (usually identical structures) using `UNION ALL`. Each table typically represents a partition of data.

Key Differences from Other View Types

- Uses multiple tables instead of one
- Often used for large datasets
- Data is horizontally partitioned

Real-Life Use Case

E-Commerce System
Sales data can be stored in separate tables by year (Sales_2023, Sales_2024, Sales_2025), and a partitioned view combines them into one view.

Example:
A view that shows all sales records across multiple yearly tables.

Limitations and Performance Considerations

- Complex to manage and maintain
- Performance depends on correct partitioning
- Requires CHECK constraints for query optimization
- Not as flexible as built-in table partitioning

Summery Part 1

View Type	What is it?	Key Difference	Limitations / Performance
Standard View	A saved SELECT query that shows data from tables	Does **not store data**	No performance improvement, can be slow with large data
Indexed View	A view with an index that **stores the result**	Data is stored physically	Slower inserts/updates, uses extra storage, strict rules
Partitioned View	A view that combines multiple tables using `UNION ALL`	Data split across many tables	Hard to manage, performance depends on design

1. Can We Use DML (INSERT, UPDATE, DELETE) on Views?

Yes, but not on all views and not always

View Type	INSERT	UPDATE	DELETE
Standard View	Yes	Yes	Yes
Indexed View	No (directly)	No	No
Partitioned View	Limited	Limited	Limited

Restrictions & Limitations When Using DML on Views

Type of View	Restriction / Limitation	Explanation
Standard View	Single base table only No JOINs No aggregate functions No GROUP BY No DISTINCT No calculated columns No UNION / UNION ALL	- DML works only if the view is based on one table JOINs usually prevent INSERT or UPDATE SUM, COUNT, AVG block DML Grouped results cannot be modified DISTINCT removes row identity Expression-based columns cannot be updated UNION views do not support normal DML
Indexed View	Direct DML not allowed	Must modify the base tables instead
Partitioned View	Requires CHECK constraints	SQL Server must know which table to modify
All Views	WHERE clause must identify rows	UPDATE/DELETE must target specific rows

Real-Life Example (Updating a View)

Example: HR System

An HR manager updates employee contact details using a view instead of the main table.

Why useful?

- Hides sensitive columns (salary, national ID)
- Allows safe updates
- Improves security

SQLQuery1.sql - A...ANOUD\anoud (55))*

```sql
create database ViewTaskDB

CREATE TABLE Employees (
    EmpID INT PRIMARY KEY,
    EmpName VARCHAR(50),
    Phone VARCHAR(20),
    Department VARCHAR(30)
)

INSERT INTO Employees (EmpID, EmpName, Phone, Department)
VALUES
(2, 'Jane Smith', '987-654-321', 'Finance'),
(3, 'Ali Khan', '555-444-333', 'IT')


CREATE VIEW EmployeeContactView
AS
SELECT EmpID, EmpName, Phone
FROM Employees

UPDATE EmployeeContactView
SET Phone = '95166166'
WHERE EmpID = 2

SELECT * FROM EmployeeContactView
```

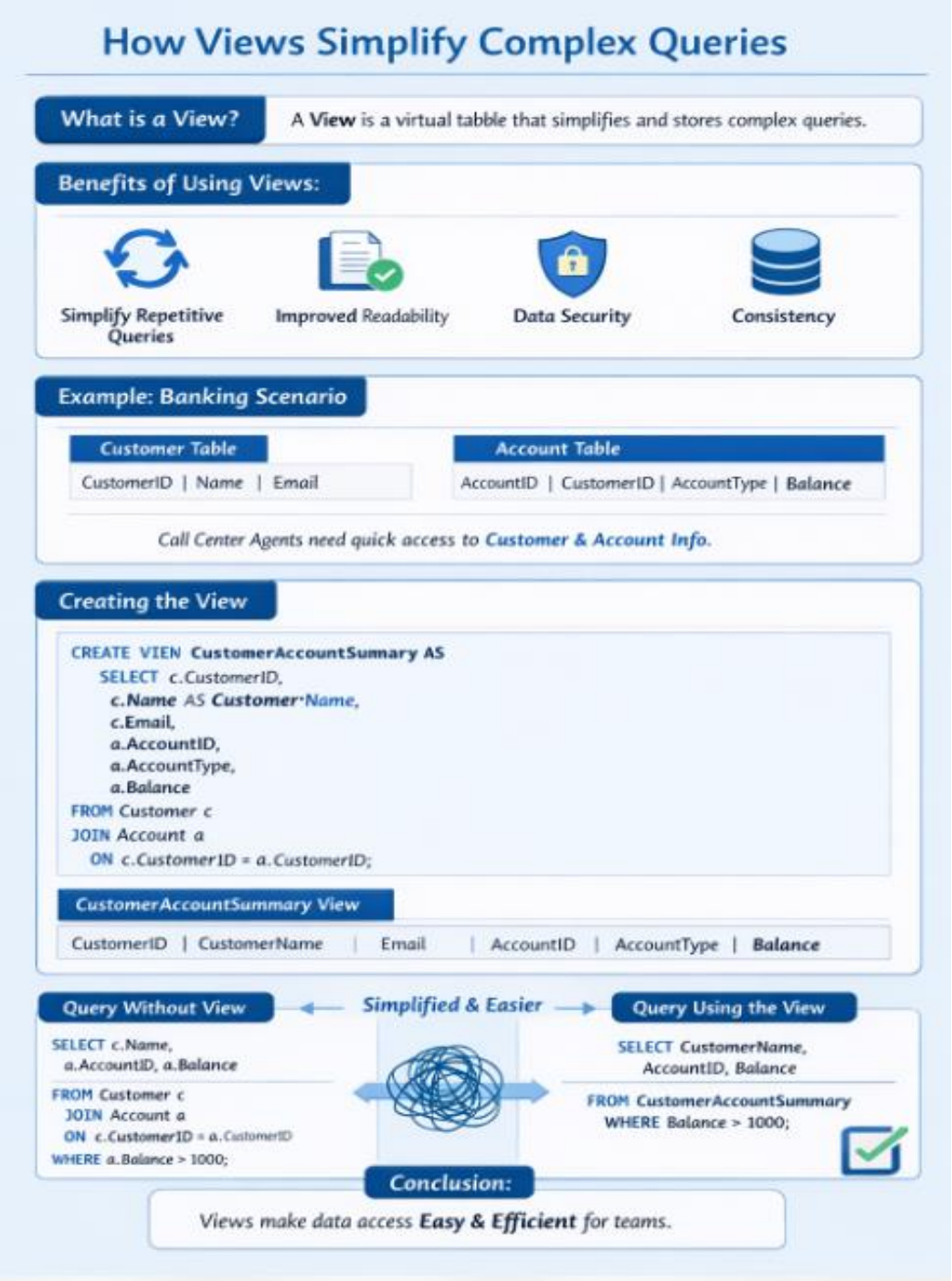
89 %

Results Messages

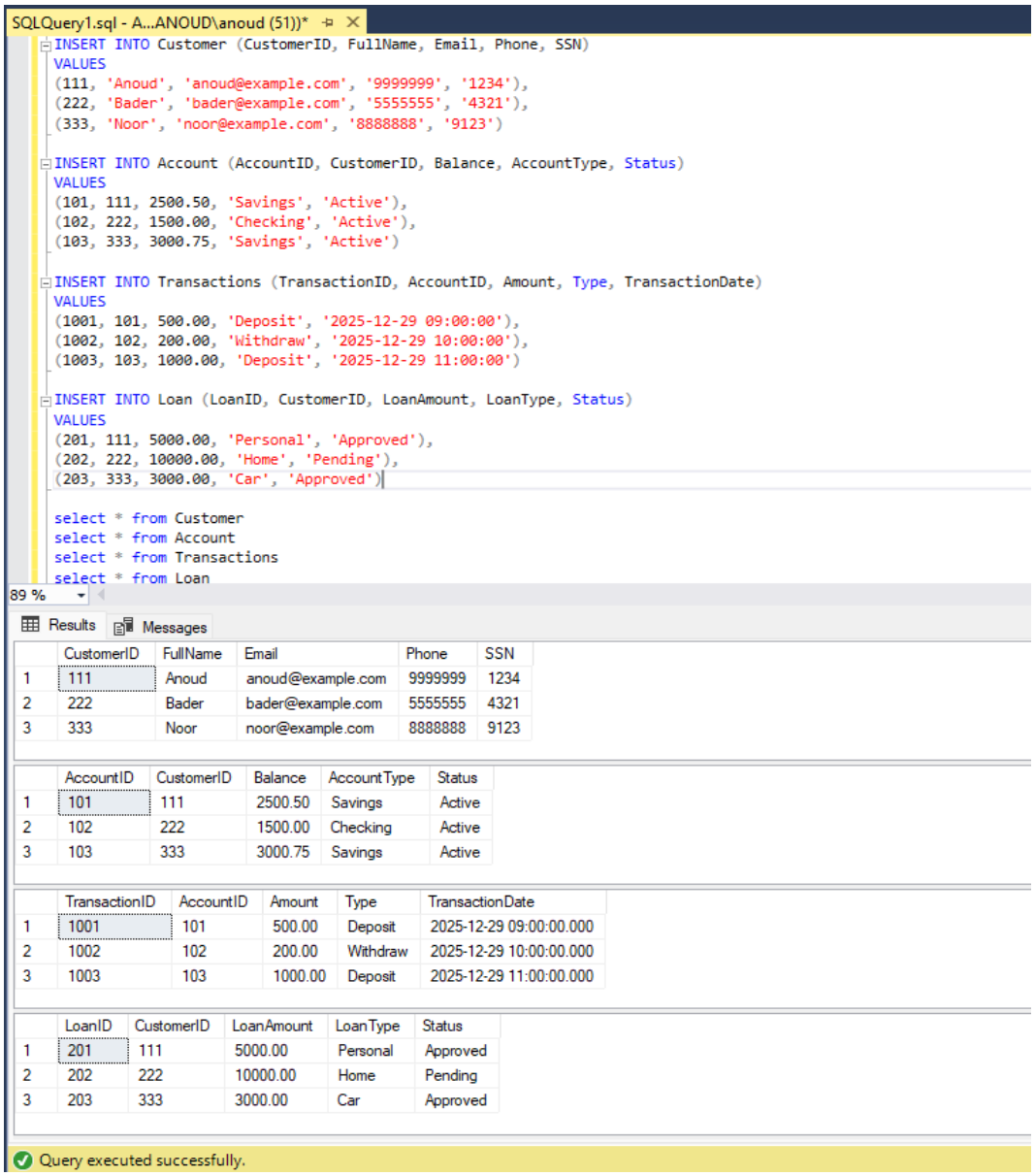
	EmpID	EmpName	Phone
1	2	Jane Smith	95166166
2	3	Ali Khan	555-444-333

2. How Can Views Simplify Complex Queries?

- Explain how a View can help simplify JOIN-heavy queries.
- Create an example view that joins at least two of your banking tables, such as:
  - Customer + Account
  - Account + Transaction
- Show how using the view reduces the need to repeat long queries.



Part 2: Real-Life Implementation Task (Banking System)



```
SQLQuery1.sql - A...ANOUD\anoud (51))*
INSERT INTO Customer (CustomerID, FullName, Email, Phone, SSN)
VALUES
(111, 'Anoud', 'anoud@example.com', '9999999', '1234'),
(222, 'Bader', 'bader@example.com', '5555555', '4321'),
(333, 'Noor', 'noor@example.com', '8888888', '9123')

INSERT INTO Account (AccountID, CustomerID, Balance, AccountType, Status)
VALUES
(101, 111, 2500.50, 'Savings', 'Active'),
(102, 222, 1500.00, 'Checking', 'Active'),
(103, 333, 3000.75, 'Savings', 'Active')

INSERT INTO Transactions (TransactionID, AccountID, Amount, Type, TransactionDate)
VALUES
(1001, 101, 500.00, 'Deposit', '2025-12-29 09:00:00'),
(1002, 102, 200.00, 'Withdraw', '2025-12-29 10:00:00'),
(1003, 103, 1000.00, 'Deposit', '2025-12-29 11:00:00')

INSERT INTO Loan (LoanID, CustomerID, LoanAmount, LoanType, Status)
VALUES
(201, 111, 5000.00, 'Personal', 'Approved'),
(202, 222, 10000.00, 'Home', 'Pending'),
(203, 333, 3000.00, 'Car', 'Approved')

select * from Customer
select * from Account
select * from Transactions
select * from Loan
```

89 %

Results | Messages

	CustomerID	FullName	Email	Phone	SSN
1	111	Anoud	anoud@example.com	9999999	1234
2	222	Bader	bader@example.com	5555555	4321
3	333	Noor	noor@example.com	8888888	9123

	AccountID	CustomerID	Balance	AccountType	Status
1	101	111	2500.50	Savings	Active
2	102	222	1500.00	Checking	Active
3	103	333	3000.75	Savings	Active

	TransactionID	AccountID	Amount	Type	TransactionDate
1	1001	101	500.00	Deposit	2025-12-29 09:00:00.000
2	1002	102	200.00	Withdraw	2025-12-29 10:00:00.000
3	1003	103	1000.00	Deposit	2025-12-29 11:00:00.000

	LoanID	CustomerID	LoanAmount	LoanType	Status
1	201	111	5000.00	Personal	Approved
2	202	222	10000.00	Home	Pending
3	203	333	3000.00	Car	Approved

Query executed successfully.

Part 3: View Creation Scenarios

1. Customer Service View

• Show only customer name, phone, and account status (hide sensitive info like SSN or balance)

```
create view CustomerServiceView As

select c.FullName AS CustomerName,c.Phone,a.status AS Account_status
from Customer c
Join Account a
On c.CustomerID = a.CustomerID


SELECT *
FROM CustomerServiceView
WHERE Account_status = 'Active'
```

89 %

Results Messages

	CustomerName	Phone	Account_status
1	Anoud	9999999	Active
2	Bader	5555555	Active
3	Noor	8888888	Active

2. Finance Department View

• Show account ID, balance, and account type

```
CREATE VIEW FinanceDepartmentView AS
SELECT
    AccountID,
    Balance,
    AccountType
FROM Account

SELECT * FROM FinanceDepartmentView
WHERE AccountType = 'Savings'
```

89 %

Results Messages

	AccountID	Balance	AccountType
1	101	2500.50	Savings
2	103	3000.75	Savings

3. Loan Officer View

• Show loan details but hide full customer information. Only include CustomerID

```
CREATE VIEW LoanOfficerView AS
SELECT
    LoanID,
    CustomerID,
    LoanAmount,
    LoanType,
    Status AS LoanStatus
FROM Loan


SELECT *
FROM LoanOfficerView
WHERE LoanStatus = 'Approved'
```

89 %

Results Messages

	LoanID	CustomerID	LoanAmount	LoanType	LoanStatus
1	201	111	5000.00	Personal	Approved
2	203	333	3000.00	Car	Approved

4. Transaction Summary View

• Show only recent transactions (last 30 days) with account ID and amount.

```
CREATE VIEW TransactionSummaryView AS
SELECT
    AccountID,
    Amount,
    TransactionDate
FROM Transactions
WHERE TransactionDate >= DATEADD(DAY, -30, GETDATE());

SELECT *
FROM TransactionSummaryView
WHERE AccountID = 101
```

89 %

Results Messages

	AccountID	Amount	TransactionDate
1	101	500.00	2025-12-29 09:00:00.000